

Danh sách tính năng Postman / Newman đã sử dụng

HW06 · 23127195 — theo yêu cầu §6 của đề bài: *"Exercise as many Postman features as you reasonably can... List the Postman features you used in your report."*

Mỗi mục dưới đây kèm **vị trí bằng chứng** trong repo để đối chiếu.

A. Đã dùng thật trong bài

#	Tính năng	Dùng vào việc gì	Bằng chứng
1	Collection (schema v2.1.0)	3 collection chính (mỗi API một collection) + 3 collection baseline cho CI + 3 collection data-driven = 9 collection	postman/collections/
2	Folder trong collection	Nhóm test case theo kỹ thuật kiểm thử (01 - Domain Partition , 04 - State Transition , 05 - Security , 06 - Schema Validation) — cấu trúc thư mục phản ánh trực tiếp cấu trúc báo cáo	Mục item[] lồng nhau trong mọi collection
3	Environment	Tách cấu hình khỏi collection: baseUrl , studentId , tài khoản mẫu, email fixture	postman/environments/eshop-local.postman_environment.json
4	Collection variables	Biến runtime ghi bởi test script: userToken , adminToken , formulaToken , couponToken , countBefore , bulkRows	pm.collectionVariables.set(...) trong test script
5	Dynamic variables	{{ \$guid }} sinh X-Run-Id cho mỗi request; {{ \$randomStreetAddress }} sinh dữ liệu payload lớn	Pre-request script cấp collection; TC-A1-023
6	Pre-request script cấp Collection	Gắn header X-Student-Id cho mọi request + console.log làm bằng chứng §11	Khởi event[listen=prerequest] ở cấp collection
7	Test script cấp Collection	Hai assertion áp cho mọi request: header đúng định dạng, và server không rò rỉ stack trace. Chính assertion thứ hai bắt được BUG-A3-09	Khởi event[listen=test] ở cấp collection
8	Pre-request script cấp Request	Sinh 100 dòng dữ liệu cho phép thử import hàng loạt (TC-A3-009)	TC-A3-009
9	Test script cấp Request	746 assertion trên 241 request	Mọi request
10	<code>pm.response.to.have.jsonSchema</code>	Kiểm tra lược đồ JSON của response thành công và response lỗi	Nhóm Schema Validation của cả 3 API
11	<code>pm.sendRequest</code> (request lồng)	Đọc lại tài nguyên để kiểm chứng độc lập — không tin vào message trả về	TC-A1-026 ... TC-A1-030 , TC-A3-032 ...
12	Request chaining	Đăng nhập → bắt token → dùng cho các request sau; import → đếm lại số bản ghi	Các request [SETUP]
13	Data-driven run (Collection Runner + data file)	3 collection chạy lặp theo file CSV — 39 iteration	postman/data/*.csv + dd1/dd2/dd3
14	<code>pm.iterationData</code>	Đọc giá trị và kỳ vọng từ từng dòng CSV — thêm phân vùng mới chỉ cần thêm một dòng dữ liệu	Test script của dd1 , dd2 , dd3
15	Postman Console	Xem log [X-Student-Id] ... theo thời gian thực — bằng chứng bắt buộc §11	newman/*.console.log ; ảnh chụp: evidence/

#	Tính năng	Dùng vào việc gì	Bằng chứng
1 6	Newman CLI	Thi hành tự động ngoài GUI	<code>scripts/run_newman.sh</code>
1 7	Newman reporter htmlextra	Báo cáo HTML kèm console log (<code>--reporter-htmlextra-logs</code>)	<code>newman/*.html</code>
1 8	Newman reporter json	Đầu vào cho <code>export_testcases.py</code> và <code>build_baseline.py</code>	<code>newman/*.json</code>
1 9	Newman reporter junit	Định dạng chuẩn để CI hiển thị kết quả test	<code>newman/*.xml</code>
2 0	--env-var ghi đè lúc chạy	CI ghi đè <code>baseUr1</code> mà không cần sửa file environment	Workflow bước 8–10
2 1	--iteration-data	Chạy data-driven từ dòng lệnh	Workflow bước 10
2 2	Newman trong CI/CD	GitHub Actions hai tầng	<code>.github/workflows/newman-23127195.yml</code>
2 3	Xuất/nhập collection dạng JSON	Collection được sinh ra từ nguồn test case rồi import vào Postman desktop	<code>postman/scripts/build_collections.py</code>
2 4	Mô tả (description) cho request	Mỗi request mang: kỹ thuật, tham số/phân vùng, kỳ vọng theo đặc tả, nguồn AI/HUMAN, nhãn audit — đọc được ngay trong GUI Postman	Trường <code>request.description</code>

B. Tính năng cần tài khoản Postman — hướng dẫn để sinh viên tự làm

Ba tính năng dưới đây yêu cầu đăng nhập tài khoản Postman và thao tác trên giao diện, nên không tạo được từ dòng lệnh. Nếu làm, chụp màn hình và lưu vào `evidence/` .

#	Tính năng	Cách làm	Gợi ý dùng cho việc gì
2 5	Work space	Postman → <i>Workspaces</i> → <i>Create Workspace</i> → đặt tên <code>HW06-23127195</code>	Nhóm 9 collection + environment vào một chỗ; chụp màn hình workspace
2 6	Mock Server	Chọn collection → ... → <i>Mock collection</i>	Tạo mock cho <code>POST /api/apply-coupon</code> trả về response đúng theo đặc tả (<code>discount_amount: 50000</code> , <code>final_amount: 450000</code>). Đây là minh hoạ rất tốt: cùng bộ test chạy xanh trên mock nhưng đỏ trên SUT thật → chứng minh test đúng, hệ thống sai
2 7	Monitor	Chọn collection → ... → <i>Monitor collection</i>	Đặt lịch chạy collection baseline mỗi giờ. Lưu ý: monitor chạy trên hạ tầng Postman nên không gọi được <code>localhost</code> — cần Postman Agent hoặc SUT có địa chỉ công khai

C. Tổng kết

24/27 tính năng đã dùng và có bằng chứng trong repo. Ba tính năng còn lại (Workspace, Mock Server, Monitor) cần tài khoản Postman — hướng dẫn thực hiện ở mục B.

Ghi chú về cách xây dựng collection. Các collection trong bài **không** được dựng bằng cách click tay từng request trong GUI, mà được **sinh ra** từ một nguồn test case duy nhất (`testcases/*.json`) bằng `build_collections.py` , rồi import vào Postman desktop. Lý do: với 241 request, dựng tay vừa dễ sai vừa không thể rà soát; quan trọng hơn, hai khiếm khuyết của bộ test (xem `ai/AI_CRITIQUE.md`) được sửa **một lần** trong bộ sinh và áp cho toàn bộ 746 assertion — điều không làm được nếu sửa tay từng request. Kết quả vẫn là file `.postman_collection.json` chuẩn v2.1, import và chạy bình thường trong GUI.